

操作系统 · 薄弱点背 诵手册

408 操作系统 · 错题血点地图

用途：每天早读 10 分钟过一遍，每条都配记忆钩子，背熟一条在打卡表记一笔。

来源：王道小程序操作系统错题 97 道（概述 27 / 进程管理 34 / 内存管理 34 / 文件系统 2）暴露的薄弱模型。

错因：概念不清 59 · 思路错误 22 · 理解偏差 15 · 计算错误 1。病因是**概念边界没划清**，不是不会算。

创建：2026-09-10 · 随对话复习暴露的新薄弱点持续追加。

背诵法则：不背"这道题选什么"，只背 "**这类题的判断式 + 记忆钩**"。

0.1 三级调度（谁管哪一步转换）

调度	别名	管的状态转换	频率	记忆钩
高级调度	作业调度	外存后备队列调入内存， 创建态 → 就绪态	最低	招人进公司
中级调度	内存调度	就绪/阻塞 ↔ 挂起态 （换入换出）	中	让员工回家待岗
低级调度	进程调度	就绪态 → 运行态	最高（几十 ms 一次）	派活给谁干

🔗 **记忆钩：**问“创建态转就绪态由谁完成”→ 答高级调度。低级调度只管就绪↔运行，中级只管挂起。

0.2 进程状态转换（只有这四条箭头合法）

转换	触发条件	记忆钩
就绪 → 运行	被 调度 选中	唯一靠调度实现的转换
运行 → 就绪	时间片用完 / 被抢占	不是变阻塞！
运行 → 阻塞	主动 请求 I/O、等事件、P 操作失败	唯一“自愿”的转换
阻塞 → 就绪	唤醒 ：I/O 完成、V 操作、资源到位	不能直接阻塞→运行

⚠ **陷阱：**三个最常错的箭头：①时间片用完 → 就绪态（不是阻塞）；②I/O 完成时 → 阻塞变就绪（I/O 发起时才是运行变阻塞）；③申请 CPU 得不到 → 仍是就绪态（阻塞只用于等 I/O 等事件，不等 CPU）。

🔗 **记忆钩：**唤醒 = 重新拿回竞争 CPU 的资格。唤醒不改优先级、不插到就绪队首、更不等于立刻运行。四条箭头里只有"运行→阻塞"是进程自己决定的，其余全被动。

0.3 单 CPU 下队列最多几个（送分题，别丢）

问法	答案	理由
n 个进程，就绪队列最多	n - 1	至少 1 个在 CPU 上运行
n 个进程，阻塞队列最多	n	死锁时全部阻塞（CPU 可空闲）
10 个进程，就绪队列最多	9	同上

⚠ **陷阱：**就绪最多 $n-1$ 、阻塞最多 n ——两个数字不一样，因为 CPU 可以闲着但进程不会全就绪。

0.4 分页 vs 分段 vs 段页式（ch3 全部混淆的总根源）

维度	分页	分段	段页式
划分者	系统 （硬件定死页大小）	用户编程时 按逻辑功能划分	用户先分段，系统段内再分页
用户可感知	不可感知 （透明）	可感知 （段是逻辑单位）	可感知段
碎片	内部碎片 （页内剩余）	外部碎片 （段间剩余）	内部碎片
引入目的	提高内存利用率	方便编程、共享保护、动态链接增长	兼取两者长处
利于动态链接	不利	有利 （按逻辑模块装配）	有利
地址转换查表	页表（硬件完成）	段表（硬件完成）	先查段表再查页表
管理原则	—	—	分段落用户地址空间，分页管物理存储空间

🔗 **记忆钩：**页是机器的格子，段是人的意思。所以：分页管物理、分段管逻辑；分页产生内部碎片、分段产生外部碎片；问"利于动态链接"选分段，问"提高内存利用率"选分页。

⚠️ **陷阱：**引入段式不是为了"提高内存利用率"（分段有外部碎片，利用率反而不如分页）——这是原题考过的反直觉点。

1.1 用户态 ↔ 内核态（15 题全在这张表）

- **用户态 → 内核态的唯一途径 = 中断**（广义中断：外中断、异常、陷入/访管指令）
- 内核态 → 用户态：执行一条特权指令（修改程序状态字 PSW）
- ⚠ **中断处理程序不是转换途径**——它是转换完成之后才在内核态执行的程序

特权指令（只能内核态）	非特权指令（用户态可执行）
开中断、关中断、置时钟、停机、清内存、设置中断屏蔽字、修改/加载 PSW、I/O 指令	算术运算、寄存器清零、存数取数、转移指令、 陷入指令本身

⚠ **陷阱**：最大陷阱——陷入(trap)指令在用户态执行。它是用户程序主动触发、把 CPU“送进”内核态的那条指令，所以它本身必须能在用户态跑。同理“转移指令”也是普通指令。

运行在用户态	运行在内核态
命令解释程序(shell) 、 用户应用程序	缺页处理程序、进程调度程序、时钟中断处理程序、中断处理程序、设备驱动程序、磁盘调度程序、OS 初始化程序

🔗 **记忆钩**：shell 是用户程序（OS 提供给用户的接口程序）。反过来，凡名字里带“处理/调度/驱动/中断/初始化”的，一律内核态。

- **不可能在用户态发生的事件 = 进程切换**（调度程序运行在内核态）。系统调用发起、外中断响应、缺页都可以在用户态发生。
- 特权指令只允许 **操作系统** 使用（不是“管理员”——管理员是人，绕不过硬件态位）

1.2 中断：谁保存什么（统考真题反复考）

内容	谁保存	方式
PC、PSW（断点信息）	硬件（中断隐指令）	自动
通用寄存器（现场）	操作系统（中断服务程序）	软件压栈
TLB、Cache	不需保存	硬件自管

🔗 记忆钩：硬件存"断点"，软件存"现场"。问"处理外中断时应由 OS 保存的是"→ 答通用寄存器的内容。

- 中断处理程序属于 **操作系统程序**（内核组成部分），不是用户程序、也不是独立第三方程序
- **时钟中断服务程序更新三项全部**：内核时钟变量、当前进程占用 CPU 时间、当前进程时间片内剩余执行时间

🔗 记忆钩：凡"跟时间有关的信息"，时钟中断都管——看到 I II III 全是时间相关的，直接选全对。

- **OS 必须提供的功能 = 中断处理**（GUI 可无、编译器可无、系统调用是接口但没中断处理更底层）
- **系统调用只能通过用户程序间接使用**（程序中调库函数 → 陷入指令）。它不是命令接口的命令、不能键盘直接用。

🔗 记忆钩：命令接口 = 人敲键盘；系统调用 = 程序里写代码。两条路，别混。

1.3 操作系统结构四兄弟

结构	特点	缺点	记忆钩
分层式	各层 单向依赖/单向调用、结构清晰 便于调试验证、易增删替换某层	请求逐层传递，效率低	楼房：只能踩下面那层
模块化	支持 动态加载 新模块、模块间 可直接调用无须消息传递（效率高）	模块相互依赖，比分层更难调试	乐高：能热插拔
微内核	只留最底层机制，高层服务放用户态服务器，可靠性高	频繁切换态，性能开销大	极简内核+外包服务
宏内核	所有功能都在内核态	结构不清晰	大而全

⚠ 陷阱：两个反直觉点：①"结构清晰且便于调试" → 选分层式（不是微内核）；②模块化 OS 的模块都在内核态，某个模块出错仍会崩内核——"模块出错不影响整机"是微内核的优点，不是模块化的。

- 分层式 不具有 "非常灵活的依赖关系"（依赖是严格受限的单向）
- 微内核里不宜放：文件系统服务（高层服务，应放用户态服务器）
- 微内核 应放：进程间通信机制、低级 I/O、低级进程管理与调度、中断和陷入处理

🔗 记忆钩：微内核只留"通信 + 低级 I/O + 低级调度 + 中断陷入"四件底层机制，文件系统这种大块头赶出去。

- 外核(exokernel)：把未经抽象的硬件资源直接分配给进程、用户进程通过 LibOS 库请求服务、减少映射开销提升效率；⚠ 外核不负责进程调度（高层管理下放给用户态 LibOS）

1.4 引导与虚拟机

- OS 引导程序(boot loader) 存放在硬盘（MBR/引导扇区），不在 BIOS

🔗 记忆钩：BIOS 是发起者（固化在 ROM），引导程序是接力棒（住在硬盘）。
开机顺序：BIOS 自检 POST → 读硬盘 0 扇区引导程序入内存 → 加载 OS 内核。

- 引导程序 只加载内核，不会把硬盘上的 OS"全部"加载进内存

- 双系统时引导程序会为用户交互选择启动哪个系统
- **虚拟机**：软件、硬件 **都能实现**；多台虚拟机同跑在一台物理机上是 **分时并发（逻辑并行）**，不是真正的并行

🔗 **记忆钩**：单 CPU 上没有"真并行"，只有"看起来同时"。

1.5 多道程序与并行

- **引入多道程序设计的前提 = 系统具有中断功能**（靠中断实现程序切换与并发控制）

⚠ **陷阱**：不是"分时功能"——分时是多道技术的应用/结果，不是前提；也不要求多 CPU、多终端。

- **多道程序基本特征**：多道、宏观并行微观串行、中断性、共享性、制约性；⚠ **没有"顺序性"**（顺序性是单道批处理的特征）
- 单处理机系统中 **可并行** 的是：处理机与设备、处理机与通道、设备与设备；⚠ **进程与进程只能并发不能并行**（单 CPU 同时只跑一个）

🔗 **记忆钩**：并行要"真的同时"，单 CPU 上只有 CPU 和它的 peripherals 能真同时；进程之间永远叫并发。

- OS 不必提供编译器、运行程序不必"预订运行时间"（靠调度算法分配）

2.1 进程 vs 程序 vs 进程映像

- **进程映像(进程实体) = PCB + 程序 + 数据** (缺 PCB 就不完整)
- 程序 = 静态指令集合; 进程 = 程序的 **一次执行过程** (动态)

⚠ **陷阱:** 把进程说成"多程序环境中的完整程序" = 混淆进程与程序, 这是"最不符合 OS 对进程理解"的典型错误说法。

- 进程与程序的四条关系 (判断正误):
- 一个进程生命周期中 **可先后执行多个程序** (exec 替换) → 对
- 一个进程 **同一时刻只能执行一个程序** (一个 PC) → 说"可同时执行多个"是 **错**
- 一个程序 **多次运行 → 多个不同进程** → 对
- 一个程序 **一次执行可产生多个进程** (fork 子进程) → 对

🔑 **记忆钩:** 时间上可换 (先后执行多个程序), 空间上不可分 (同一时刻只跑一个)。

2.2 进程的创建与撤销

- **进程创建原语做三件**: 申请并初始化 PCB、为新进程分配资源 (内存空间)、把新进程插入就绪队列

⚠ **陷阱:** 创建原语不分配 CPU——分配 CPU 是低级调度 (进程调度) 的职责。

- **会创建新进程的情形 (四项全都会)**: 用户登录、高级调度发生、OS 响应用户请求、用户打开一个程序 (浏览器)
- **用户进程随其运行正常或异常结束而被撤销** (不是常驻等操作员撤销; 时间片轮转、阻塞唤醒只是状态转换, 进程依然存在)

2.3 并发进程的特性

- 并发进程运行结果 **具有不可再现性**（失去封闭性）——因为相对速度不可预知
- 进程生命周期是 **间断的**（有就绪、等待），不是连续的
- 对共享变量的操作结果 **与执行速度有关**

2.4 线程

- 线程 **有自己的 CPU 现场**（寄存器、PC、栈），**可独立执行** 程序 → 这是线程的正确描述
- 同进程内各线程 **共享该进程的地址空间**（说"每个线程有独立地址空间"是错的）
- 一个进程 **可含多个线程**
- 同进程线程间通信 **可直接读写共享内存，不必使用系统调用**
- **同进程内线程切换只需更新：PC + 栈基址寄存器**

🔗 **记忆钩：**页表基址寄存器和进程打开文件表是"进程级"资源，同进程内切换不变。判断诀窍：问"这东西是线程私有还是进程共享"——PC 和栈是线程私有（要换），页表和文件表是进程共享（不换）。

- **多线程系统的特长：**并行计算（矩阵乘法）、并发响应多个请求（Web 服务器）、GUI 中并发处理输入/计算/跟踪

⚠ **陷阱：**⚠ 键盘驱动为每个应用配一个线程不是特长——键盘输入是串行、低速、单一输入源，配多线程纯浪费。判断诀窍：场景本身能不能"同时发生"，不能并行的活配多线程无意义。

2.5 进程上下文与调度细节

- **进程上下文包括：**进程现场信息（寄存器、PC 等 CPU 状态）、进程控制信息（PCB 管理信息）、用户堆栈

⚠ **陷阱：**⚠ 中断向量不属于进程上下文——它是系统中断机制的公共表项，与具体进程无关。

- **处于临界区的进程仍可能被调度切换出去**（时间片用完照样被剥夺 CPU）

🔗 **记忆钩：**临界区只管"互斥进入"，不管"禁止调度"。别的进程不抢同一临界区就行。

- 上下文切换是进程调度的 **实现手段**；进程调度的目标是让进程轮流使用处理器

2.6 调度算法六连问（必背表）

算法	抢占性	会不会饥饿	利于谁	记忆钩
先来先服务 FCFS	非抢占	不饥饿	利于 CPU 繁忙型，不 利于 I/O 繁忙型	排队买票，长队拖死短客
短作业/短进程优先	抢占、非抢占 两版都有	长作业会 饥饿	短作业	谁快谁先走
高响应比优先	非抢占	不饥饿	既体现短作业优先又不饥饿	$\text{响应比} = (\text{等待} + \text{服务}) / \text{服务}$
时间片轮转 RR	绝对可抢占	不饥饿	交互式、I/O 型	到点必让位
优先级调度	抢占、非抢占 两版都有	低优先级会饥饿	紧急作业	—

🔗 **记忆钩：**"绝对可抢占"只有时间片轮转——因为到点强制让位是它的定义。优先级和短进程优先都有两种版本，说"绝对"就不成立。

🔗 **记忆钩：**高响应比 = 短作业优先 + 不饥饿的唯一答案。公式： $\text{响应比} = (\text{等待时间} + \text{要求服务时间}) / \text{要求服务时间}$ 。服务时间短→比值高（照顾短作业）；等待时间长→比值也高（长作业等久了也能翻身）。

- FCFS 利于 CPU 繁忙型：**一旦拿到 CPU 就跑到完成或主动阻塞，长作业爽；I/O 型作业频繁短用 CPU 却要排队，设备利用率低。
- 降低进程优先级的合理时机 = 时间片用完**（说明它偏计算型，该让位）

- **提高优先级的时机** = 刚完成 I/O 进入就绪队列、长期处于就绪队列（防饥饿、鼓励 I/O 型）

🔗 **记忆钩：** 占用 CPU 太久 → 降级；等了太久或刚让出 CPU → 升级。"从就绪转运行"是调度发生的时刻，不是调优先级的时机。

2.7 同步与互斥（PV 操作）

- **P、V 操作是低级进程通信原语**（不可中断的原语；不是机器指令、不是系统调用命令、不是作业控制命令）
- **P 操作后进入等待队列的条件：S.value < 0**（P 先执行 value--，减完仍小于 0 说明资源已分光且有人排队）
- **互斥信号量 mutex（初值 1）的取值含义：**

mutex 值	含义
1	没有进程进入临界区
0	恰有一个进程在临界区（无人等待）
-1	一个在临界区，另一个在等待

🔗 **记忆钩：** 0 是"一人独占"，负数才代表"有人在门外排队"，绝对值 = 排队人数。

- **互斥锁：** 既可用于多线程也可用于多进程；遵循 **"谁加锁谁解锁"**（不要求是创建者）

⚠️ **陷阱：** ⚠️ 说"只能由创建它的线程加锁解锁"是错的，正确的是"只能由对它加锁的线程/进程解锁"。

2.8 死锁（三种策略别混）

策略	做法	典型算法/手段	破坏的条件
预防	破坏四个必要条件之一	一次性/静态资源分配	请求与保持
		资源有序分配 （按编号递增申请）	循环等待
		剥夺资源法	不可剥夺
避免	分配前判断会不会进入不安全状态	银行家算法	不破坏条件，只防进入 不安全状态
检测与解除	允许死锁发生，事后查出并解除	死锁检测算法、资源分配图化简法、定时运行检测程序	—

🔗 **记忆钩**：预防 = 拆条件，避免 = 看状态，检测 = 事后抓。银行家属"避免"不属"预防"——这是原题考过的最大坑（问"属于预防策略的是"→选资源有序分配，不选银行家）。

⚠️ **陷阱**：PV 操作 / 互斥使用共享资源 都不能防止死锁（只保证互斥访问）；"定时运行死锁检测程序"属检测不属预防。

- **必不死锁的最少资源数公式**：n 个进程各需 k 个同类资源 → **总数 $\geq n \times (k - 1) + 1$**

🔗 **记忆钩**：最坏情况是每个进程都差一个（各持 k-1 个），多给 1 个就能有人凑满、完成、释放。3 进程各需 4 个 → $3 \times 3 + 1 = 10$ 。

- **哲学家进餐**：限制同时允许拿起筷子的哲学家 **人数 $\leq n - 1$** → 破坏 **循环等待** 条件，预防死锁

⚠️ **陷阱**：⚠️ "使用信号量就一定可以避免死锁"过于绝对（用得不当照样死锁）；"同时检查两支筷子"会反复失败造成饥饿。

3.1 连续分配与动态分区

- **单用户连续分配不适合多道程序设计**（整个用户内存一次给唯一作业，一次只能跑一道程序）；固定分区、可变分区、分页都能容纳多道
- **动态(可变)分区在作业"装入"内存时动态建立**（不是创建时——创建时还没装入、没实际分配）
- **回收空闲区时空闲区表项数变化**：

情况	表项数变化
无上邻也无下邻空闲区	+1 （新增一项）
只有上邻 或 只有下邻	不变 （并入已有项）
既有上邻又有下邻	-1 （三项合成一项）

🔗 **记忆钩**：邻居越多，合并越狠。0 个邻居→多一项；1 个→不变；2 个→少一项。

- **基于索引搜索的动态分区分配算法**：快速适应算法、伙伴系统、哈希算法

⚠ **陷阱**：⚠ 最佳适应算法属于顺序搜索（和首次适应、最坏适应一样遍历空闲链表），不属索引搜索。

3.2 内存保护与重定位

- **多进程彼此互不干扰 → 靠"内存保护"**（界地址寄存器/越界检查）

🔗 **记忆钩**：分配只是划地盘，映射只是翻译地址，扩充只是虚存，只有"保护"才管"不许越界"。

- **动态重定位依赖三件**：可重定位装入程序、重定位寄存器、地址变换机构

⚠ 陷阱：⚠ 不依赖"目标程序"（那是链接产物，不是重定位机制的要素）。

- 重定位寄存器不是每进程一个！硬件通常只有一个，进程切换时由 OS 把当前进程的起始物理地址装进去

🔗 记忆钩：物理地址 = 逻辑地址 + 重定位寄存器的值；装入后位置可移动（改寄存器即可）；映射发生在运行时。

3.3 分页机制细节

- 每个进程一张独立页表，且页表必须常驻内存（不能共享、不能只驻留运行进程的）

🔗 记忆钩：页表要随时供地址转换和进程切换，所以常驻。

- 查页表由硬件完成（地址转换机构 + TLB），操作系统只负责建立和维护页表

🔗 记忆钩：每次访存都要查表，软件做太慢 → 硬件做。OS 是"建表的"，不是"查表的"。

- 关闭 TLB 后，每存取一条指令或一个操作数至少访存 2 次（查页表 1 次 + 取目标 1 次）
- 页式存储管理会产生内部碎片（页内剩余）
- 页面对用户不可感知（透明）；可感知的是段
- 页式采用动态重定位（经页表映射），不能用静态重定位

3.4 段式机制细节

- 一个程序如何分段，是在"用户编程时"决定的（程序员按逻辑功能划分：主程序段、子程序段、数据段）

🔗 记忆钩：段是"人的意思"，所以人写代码时就定了；编译后段结构固定，装入/执行都不改。

- 一个进程有 n 个段 → 1 张段表、n 个段表项

⚠ **陷阱：** ⚠ 段表数 = 1，段表项数 = n。问"需要几个段表"答 1。

3.5 虚拟内存基础

- **虚存两大特征：** **多次性**（运行前不必全部装入，可分多次调入）+ **对换性**（运行中不必一直驻留，可换出再换入）

🔗 **记忆钩：** 正确说法 = "运行前不必全部装入，且运行中也不必一直驻留"（两个"不必"都要）。

- **能提供虚拟存储技术的 = 请求页式、请求段式**（必须有"请求调入"机制）

⚠ **陷阱：** ⚠ 基本页式不行（无请求调页）、动态分区不行、存储覆盖不行。看到"请求"二字才是虚存。

- **请求分页的主要特点 = 扩充内存**（逻辑上把内存变大）

⚠ **陷阱：** ⚠ 不是"消除内部碎片"（分页仍有页内碎片）；"便于动态链接""便于信息共享"是分段/请求段式的特点。

- **页式虚拟存储管理中程序的链接方式必然是"运行时动态链接"**

🔗 **记忆钩：** 按需调页 = 按需装配模块，所以链接也必须推迟到运行时。

- **页表项各标志位分工：**

标志位	决定什么
有效位（存在位）	是否会发生页故障 （0 = 不在主存 → 访问即缺页）
修改位（脏位）	换出时 是否需要写回 外存
保护码	访问 权限 检查（读/写/执行）
访问位	置换算法参考（最近有没有被访问）

🔗 **记忆钩：** 有效位管"在不在"，脏位管"要不要写回"，保护码管"能不能碰"。

3.6 缺页中断处理（统考真题高频）

- 缺页处理过程中 OS 可能执行：修改页表 + 磁盘 I/O + 分配页框（三项全可能）
- ⚠ 缺页时不会处理越界错——越界检查发生在查页表之前，既然已查到页表并产生缺页，说明没越界

📌 记忆钩："分配、调页、改表"三件事都做，"越界"不做。顺序是：越界检查 → 查页表 → 有效位为 0 → 缺页中断 → 分配页框（可能要置换）→ 磁盘 I/O 调入 → 修改页表（有效位置 1、填物理块号）→ 重新执行被中断的那条指令。

- 缺页中断属"故障(trap)"类内中断 → 处理后执行"被中断的那一条"指令

📌 记忆钩：外中断执行完当前指令再响应、接着执行下一条；内中断（故障）没执行完，回来重做同一条。

- 全局置换 vs 局部置换：全局 = 可从系统 所有 页框（含其他进程的）中选一页替换；局部 = 只能在 本进程 已分配的页框内选

📌 记忆钩：全局 = 全公司抢工位，局部 = 只在自己部门内挪。

3.7 页面置换算法要点

- Belady 现象：FIFO 有，LRU 没有（分配更多页框反而缺页更多，只发生在 FIFO）

📌 记忆钩：FIFO 不讲道理才会"越大越差"；LRU 按最近使用淘汰，属于栈式算法，不会 Belady。

- 工作集页面必须都在"主存储器"内才能有效运行，否则频繁调入调出（抖动）

⚠ 陷阱：⚠ 说"都在虚拟存储器内"是错的——虚存本来就包含外存，全在虚存里等于没说。

- 缺页次数下限 = n （引用串中不同页号的个数），与用什么算法无关

📌 记忆钩：初始全空，每个不同页号第一次被访问必然缺页，所以至少 n 次。

- **改进型 CLOCK 淘汰次序：** $(0,0) \rightarrow (0,1) \rightarrow (1,0) \rightarrow (1,1)$ （A = 访问位，M = 修改位）

🔗 **记忆钩：**先看"有没有被访问"（A），再看"脏不脏"（M）——A 权重更高。所以 $(0,1)$ （没访问过但脏）排在 $(1,0)$ （访问过但干净）前面。

3.8 提高 TLB 命中率

- **有效方法：**①增大 TLB 容量 ②提高页面大小

🔗 **记忆钩：**页面变大 $\rightarrow$ 同样地址空间页数变少 $\rightarrow$ 页表项变少 $\rightarrow$ TLB 装得下更多 $\rightarrow$ 命中率上升。

- ⚠️ **多级页表对 TLB 命中率无影响；降低页面大小反而降低命中率**

3.9 内存管理计算公式（唯二的计算考点）

页内偏移位数 = $\log_2(\text{页面大小})$ 页号位数 = 虚拟地址位数 - 页内偏移位数

页表项数 = $2^{\text{页号位数}}$ 每级页表消耗位数 = $\log_2(\text{一页能装几个页表项}) = \log_2(\text{页面大小} / \text{页表项大小})$

多级页表级数 = 页号位数 $\div$ 每级消耗位数（向上取整）

- 例1：32 位虚地址、页面 512B ($=2^9$) $\rightarrow$ 页号 32-9 = **23 位** $\rightarrow$ 页表项 **2^{23} 个**（物理地址 16 位是 **干扰项**，它决定物理页框数）
- 例2：48 位虚地址、页面 4KB ($=2^{12}$)、页表项 8B $\rightarrow$ 页内偏移 **12 位**、页号 36 位；每页装 $4\text{KB}/8\text{B} = 512 = 2^9$ 项 $\rightarrow$ 每级吃 9 位 $\rightarrow 36/9 =$ **4 级页表**

⚠️ **陷阱：**⚠️ 算页表项数用虚拟地址位数，不要用物理地址位数——页表描述的是"逻辑空间有多大"。

3.10 数组访问缺页数（两道统考风格计算题）

每页能装的行数 = 页框大小 $\div$ (每行字节数); 按列访问行优先存储的数组 = 最差局部性

- **判断三步**: ①算一页能装几行 ②看访问顺序是行优先还是列优先 ③列访问时每跨过"一页装的行数"就缺页一次
- 例1: `int a[128][128]` (int 4B), 页框 512B $\rightarrow$ 一页装 $512/4 = 128$ 个 int = **1 行**; 按列访问 `a[j][i]` $\rightarrow$ 每个元素都在不同行 $\rightarrow$ **每访问一个元素缺页一次** $\rightarrow 128 \times 128 = 16384$ 次
- 例2: `int X[64][64]`, 页框 128 字 $\rightarrow$ 一页装 **2 行**; 按列访问 $\rightarrow$ 每 2 个元素缺页 1 次 $\rightarrow 4096/2 = 2048$ 次

🔗 **记忆钩**: 行优先存储 + 按列访问 = 最坏组合。一页装 k 行, 则每 k 个元素缺页 1 次, 总缺页 = 元素总数 $\div$ k。

3.11 其他零散但考过的结论

- **对主存的访问以"字节或字"为单位**, 不随管理方案而变 (页/段只是 **分配和地址映射** 的单位, 逻辑记录是用户概念)

🔗 **记忆钩**: CPU 零售按字节/字, 页和段只是"分地皮的单位"。

- **可重入代码 (纯代码)** $\rightarrow$ **减少对换数量** (多进程共享内存中一份即可, 省调入调出)

⚠ **陷阱**: ⚠ 与"改变时间片长度""改变用户数""提高对换速度"都无关。

- **内存映射文件**: 按需加载部分页 (不是一次性全加载)、**支持写操作** (改内存即改文件)、多进程可通过页表映射到同一物理内存 **共享同一文件**

- **内存索引节点特有、磁盘索引节点没有的 = 访问计数值**（每有一个进程访问该索引节点 +1，访问结束 -1）

📌 **记忆钩：**计数值是"运行时状态"，只有装进内存才有意义；文件物理地址、长度、类型都是文件的静态属性，磁盘 inode 里就有。

- **文件的逻辑结构 = 流式文件（无结构文件）+ 有结构文件**

⚠️ **陷阱：**⚠️ 连续文件、链接文件属于物理结构；系统文件是按用途分类——三者都不是逻辑结构。

📌 **记忆钩：**逻辑结构 = 用户眼里文件长什么样（流式/记录式）；物理结构 = 磁盘上怎么摆（连续/链接/索引）。

#	陷阱	正确结论
1	以为陷入指令是特权指令	陷入指令在用户态执行（它是进入内核态的触发器）
2	以为时间片用完变阻塞	变就绪态
3	以为 I/O 完成时运行→阻塞	I/O 完成 是 阻塞→就绪；发起 才是运行→阻塞
4	以为引导程序在 BIOS	在硬盘 MBR，BIOS 只是发起者
5	以为微内核结构清晰便于调试	"结构清晰便于调试"是分层式；微内核的卖点是可靠性高
6	以为银行家算法属死锁预防	属死锁避免；预防的代表是资源有序分配 / 一次性分配
7	以为分段提高内存利用率	分段做不到（有外部碎片）；提高利用率是分页的目的
8	以为基本页式提供虚拟存储	必须是请求页式/请求段式
9	以为缺页时会处理越界错	越界检查在查页表之前，缺页时不会处理越界
10	以为 LRU 也有 Belady 现象	只有 FIFO 有；LRU 没有

6.1 OS 四大管理功能（处存文设）— 重复错 S1

功能	管什么
处理机管理	进程/线程的创建、调度、同步互斥、死锁
存储器管理	分配回收、地址映射、共享保护、虚拟内存
文件管理	文件目录、按名存取、读写保护、磁盘空间
设备管理	设备分配、驱动、缓冲、虚拟设备

🔗 **记忆钩：**处存文设 = 处理机 / 存储器 / 文件 / 设备。干扰项多是"用户接口、作业管理"，不属四大功能。

6.2 临界区访问四原则（空忙有让）

原则	含义	防什么
空闲让进	无进程在临界区时应允许进入	提利用率
忙则等待	已有进程在临界区则门外候着	保互斥
有限等待	等待时间有限、终能进入	防 死等/饥饿
让权等待	进不去就让出 CPU、不占着空转	防 忙等 浪费

🔗 **记忆钩：**空(空闲让进) 忙(忙则等待) 有(有限等待→防死等) 让(让权等待→防忙等)。旧答"进出打招呼通知"是礼貌不是原则，错。

6.3 信号量 S 物理含义 + P/V + 初值

- **$S > 0$** : $S =$ **剩余可用资源数**（不是"正在使用的进程数"，曾说反）

- **S = 0** : 资源刚好分光、无人等待
- **S < 0** : |S| = 在该信号量队列表里 **阻塞等待的进程数**
- **P** : S = S - 1; 若 S < 0 → 阻塞入队。 **V** : S = S + 1; 若 S ≤ 0 → 唤醒队首。二者对同一信号量成对出现
- **初值规则** : **互斥信号量恒 = 1** (临界区一次一个); **同步信号量 = 0** (等对方发信号)

⚠ **陷阱** : 打印机这类互斥资源初值就是 1, 不是 -1。-1 是运行中被抢走资源时"变到"的中间值, 别把中间值当初值。

6.4 互斥 vs 同步 (同步 ≠ 同时开始)

- **互斥** : 不能 **同时** 用同一临界资源 = 间接制约 (抢同一间厕所)
- **同步** : 按约定的 **先后次序/速度** 协调 = 直接制约 (要用对方产出就等它先做完那步)

⚠ **陷阱** : 同步是"讲顺序、等对方", 不是"一起冲 / 同时开始" (旧答)。

6.5 死锁四条件 (互请剥环)

条件	含义	破坏手段 (预防)
互斥	资源独占不可共享	一般不破坏
请求与保持	已持有资源还去请求新资源	一次性静态分配
不可剥夺	已获资源不能被强制抢走	剥夺资源法
循环等待	进程首尾相连互等	资源有序分配

🔗 **记忆钩** : 互请剥环。⚠ "资源不足/资源少于进程数"是死锁的根本成因, 不是四个必要条件之一。破坏任一条件即可预防; 银行家算法属"避免"不属"预防"。

6.6 进程 vs 线程 (线程"轻"在哪)

- 进程 = **资源分配** 基本单位（独立地址空间 + 完整 PCB）；线程 = **CPU 调度** 基本单位（共享进程地址空间，私有只有 栈 + 寄存器 + PC + TCB）
- **线程切换**：不换页表、不刷 TLB/Cache，只存少量寄存器与栈 → 开销小
- **进程切换**：换页表（地址空间变了）+ 全套 PCB → 开销大；引入线程正是为减少切换开销、提高并发度

📌 **记忆钩**：同进程线程共享"地址空间"这份大资产，切换只动 PC/栈；换进程 = 换整套房产。

6.7 三级调度 + 为何低级调度必有（详见 0.1）

- 高级(作业)：选后备作业进内存建进程（批处理、频率最低，现代系统常无）
- 中级(内存)：换出/换入到对换区（提高内存利用，非必有）
- 低级(进程)：从就绪队列选进程分配 CPU（频率最高、毫秒级）

📌 **记忆钩**：只有低级调度任何多道系统都必有——CPU 每时每刻得有人决定下一个给谁。⚠️ 调度不能发生在关中断、中断处理中、内核临界区。

6.8 页式地址转换（PTBR + f 拼 W）

1. 由 **PTBR**（页表基址寄存器）定位本进程页表
2. 页号 P 查页表第 P 项 → 得物理 **块号 f**（+ 有效位判缺页）
3. 物理地址 = f **拼接** 页内偏移 W

📌 **记忆钩**：只有 P 变 f，W 原样照搬。W 不变的根本：页大小 = 块大小，一页正好塞满一块，只换"块"不换"块内第几个字节"。

6.9 页面置换补充：OPT 不可实现 + LRU 术语

- **OPT（最佳置换）**：换掉"未来最久不用"的页 → 需 **预知未来**、**无法实现**，只作衡量基准
- **LRU** = **最近最久未使用**（往回看淘汰最久没用的），不是"未来"
- **Belady 异常**（页框变多缺页反增）**只有 FIFO**；LRU/OPT 属栈式算法绝不出现（见 3.7）

- Clock（时钟置换）：用访问位环形扫描，是对 FIFO 的改进

6.10 虚存三特征 + 缺页 Fault 重执行本条

- 虚存三特征 "多换虚"： **多次性**（分多次调入）/ **对换性**（运行中可换出换回）/ **虚拟性**（逻辑扩充容量）
- **缺页中断属"故障 Fault"类内中断**：找空闲块（无则置换，脏页先写回）→ 外存调入 → 改页表（有效位 1、填块号）+ 更新 TLB → **重新执行刚才那条指令**

🔗 **记忆钩**：外中断回来走下一条，缺页(Fault)回来重做这一条（因它没执行成功）。

6.11 位示图（页框/磁盘分配）

- 一个 **bit 对应一个物理块**：**0 = 空闲，1 = 占用**
- **块号 = 字号 × 字长 + 位号**（注意从 0 还是 1 起算，考研常在此设坑）
- 分配 = 找 0 位置 1；回收 = 清 0。直观、分配回收都快

🔗 **记忆钩**：位示图 = 一张占用格子表，0 空 1 占。

6.12 静态 vs 动态重定位

	静态重定位	动态重定位
时机	装入时一次性换算	每次访存由硬件现算
运行中能否移动	不能（地址写死）	能（改重定位寄存器即可）
是否要连续内存	要	不要
硬件机构	无	需 重定位(基址)寄存器

🔗 **记忆钩**："靠基址寄存器且支持移动/紧凑/共享"= 动态；现代分页系统用动态重定位。物理地址 = 逻辑地址 + 重定位寄存器。

批次	内容	一刷	二刷（只刷错题）
D1	零节 全局判断框架（四张表）	☐	☐
D2	一、操作系统概述 27 题（重点：用户态/内核态）	☐	☐
D3	二、进程管理 34 题（重点：调度算法六连问 + 死锁三策略）	☐	☐
D4	三、内存管理 34 题（重点：分页分段对比 + 虚存 + 计算题）	☐	☐
D5	四、文件系统 + 五、十大反直觉陷阱	☐	☐
D6	六、费曼新增血点（os-01~03：四大功能/四原则/信号量/死锁/位示图/重定位）	☐	☐

✅ 手册建于 2026-09-10，覆盖 97 道错题全部考点。⌚ 建议节奏：每天早读 10 分钟扫一节，5 天走完一轮；二刷只看"十大反直觉陷阱"表 + 各章 hook 卡片。

操作系统薄弱点背诵手册 · 2026-09-10 创建 · 随复习持续追加 · 错题本同步：网站
→ 错题本 → 408计算机 → 操作系统